

PolicyHub

UX Specification — Wireframes & Design Decisions

Version 1.0 · March 2026 · Confidential

This document covers the authoring and reader surfaces of the PolicyHub platform: a git-backed policy management system with a structured approval workflow, role-based reader portal, AI-assisted authoring, and a citation-grounded policy bot. Eight screens are documented here with annotated wireframes and the design decisions that determined each screen's structure.

Surface	Screens	Primary users
Authoring	Dashboard, Editor, Submit for Review, Review & Approval	Authors, Reviewers, Approvers
Reader	Policy Library, Policy Detail, Version History, Version Diff	All staff (role-filtered)

Cross-cutting design decisions

AI pattern — All AI features use the same FAB + floating popover pattern — the AI assistant in the editor and the Policy Bot in the portal are visually and behaviourally identical. Users learn the interaction model once.

Diff consistency — The unified diff / side-by-side tab pattern appears in both the review screen (authoring) and the version diff screen (reader). Colleagues trained on one context carry that knowledge to the other.

RBAC invisibility — Restricted content is never shown as forbidden — it simply does not exist in the UI. Missing policies return 404, not 403, and are absent from search results.

Workflow as named actions — Workflow transitions are explicit named actions (Submit, Approve L1, Approve Final, Request Changes, Resubmit) rather than generic status edits. Each action validates state, updates git, and fires notifications atomically.

Git abstraction — Users never interact with git directly. "Submit for review" opens a GitLab MR. "Approve" merges it. The git layer is completely invisible to non-technical users.

Authoring Surface

01 — Dashboard

Role: Author / Reviewer / Approver

Trigger: Default landing page after login

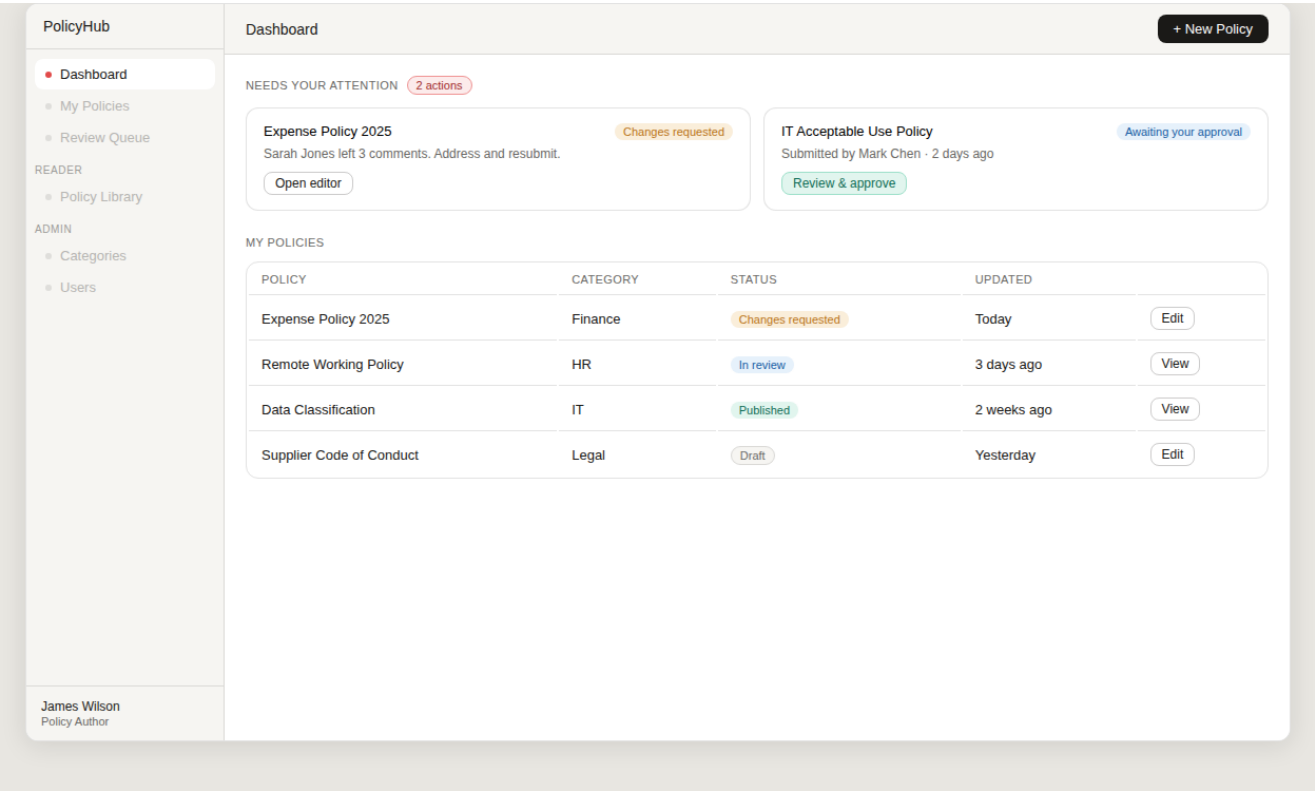


Fig. 01 — Dashboard

The command centre for all authoring-side users. Shows workflow actions requiring attention at the top — highlighted cards with direct action buttons — so the most important thing is always visible without scrolling. Below sits a paginated table of the user's own policies with inline status badges. The red pulsing dot in the sidebar is the only persistent notification signal.

Design decisions

- "Needs your attention" cards collapse complex workflow state into a single glanceable action per item.
- Attention cards always appear above the full policy list — priority ordering is structural, not configurable.
- Status badges use colour semantically: amber = action needed by me, blue = waiting on someone else, green = complete.
- New Policy button is always visible in the topbar — creating is never more than one click away.

02 — Policy Editor

Role: Author

Trigger: Clicking "Edit" on a draft or changes-requested policy

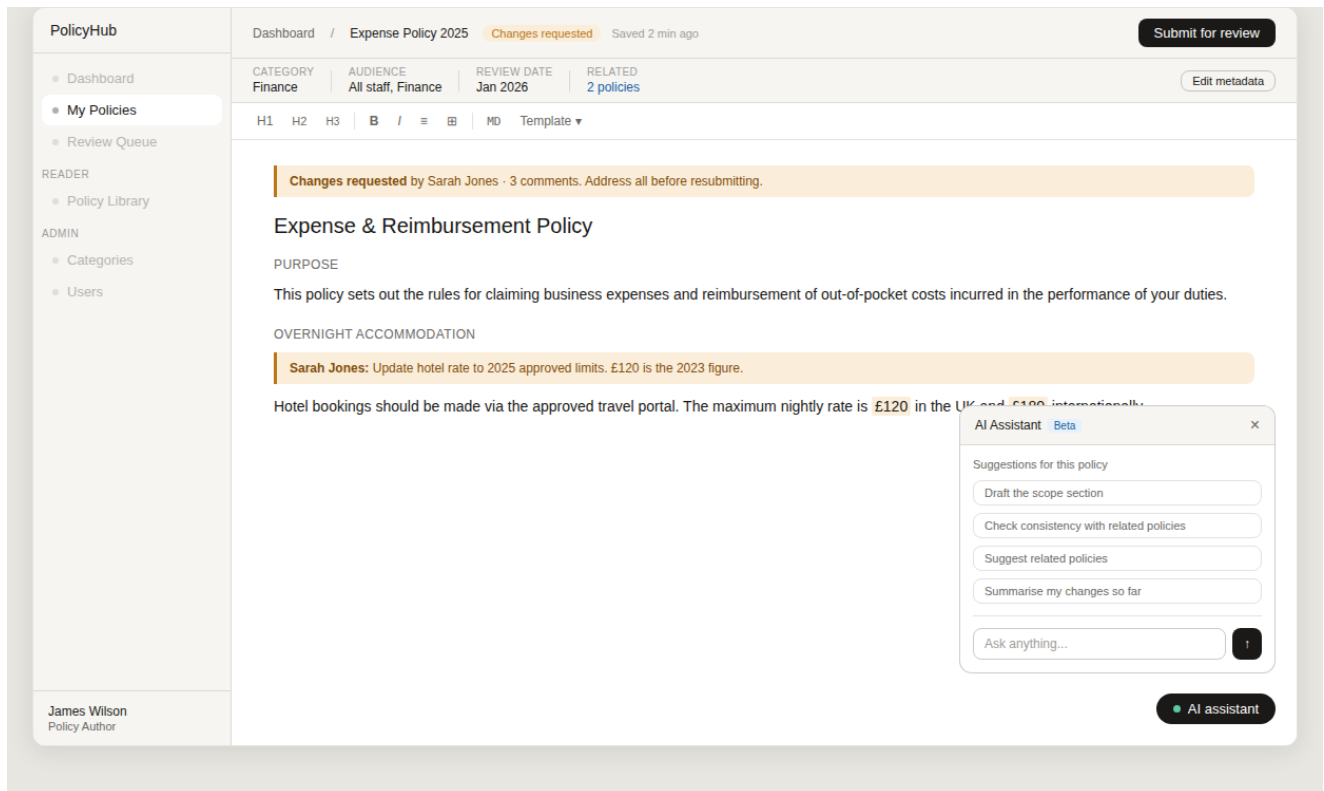


Fig. 02 — Policy Editor

The primary authoring environment. TipTap WYSIWYG editor with full rich-text toolbar plus a Markdown source toggle for technical users. A persistent metadata strip below the topbar keeps policy context (category, audience, review date) visible without consuming editor space. Reviewer comments from a "Changes requested" cycle appear inline in the document at the relevant location, with the affected text highlighted in amber. Auto-save fires every 30 seconds and on explicit save; status shown in the topbar.

Design decisions

- AI assistant is a floating popover anchored bottom-right — visible only when invoked, never competing with the document.
- Reviewer inline comments appear in document context (not a separate panel) so the author understands exactly what needs changing.
- Metadata strip is read-only in this view — a separate "Edit metadata" action prevents accidental changes during drafting.
- The toolbar includes a Template picker to load the standard policy boilerplate into a new document.
- MD toggle (top-right of toolbar) exposes raw markdown for technical users without changing the default experience.

03 — Submit for Review

Role: Author

Trigger: Clicking "Submit for review" in the editor topbar

PolicyHub Expense Policy 2025

- Dashboard
- My Policies**
- Review Queue

READER

- Policy Library

ADMIN

- Categories
- Users

James Wilson
Policy Author

Submit for review

Assign reviewers and confirm before opening the review workflow.

AI CHANGE SUMMARY — VERIFY BEFORE SENDING
Updated hotel accommodation rates for 2025: UK rate £120 - £145, international £180 - £210. Receipts requirement clarified for overnight stays.

Change magnitude Low — no justification required

Reviewer
Sarah Jones (Finance SME)

L1 Approver
Sarah Jones (Finance Manager)

L2 Approver
Policy Committee (fixed for Finance category)

[Back to editor](#) [Submit for review](#)

Fig. 03 — Submit for Review

A deliberate pause-and-confirm step before the policy enters the workflow. The AI-generated change summary is shown here first — the author verifies it is accurate before it reaches reviewers. Change magnitude is shown as a visual bar; low magnitude requires no action, high magnitude expands a required justification field. Approver assignment follows category defaults but can be overridden. The L2 approver (committee) is locked per category and shown as read-only to make the governance structure explicit.

Design decisions

- AI change summary shown to the author before submission — they verify accuracy, not the reviewer.
- Change magnitude guardrail is soft: it warns and requires justification above a threshold, but never blocks submission.
- L2 approver is non-editable by default (set at category level by admins) — this enforces governance structure.
- The back button preserves editor state — submission is cancellable without losing edits.

04 — Review & Approval

Role: Reviewer / L1 Approver / L2 Approver

Trigger: Clicking a policy in the Review Queue, or "Review & approve" from Dashboard

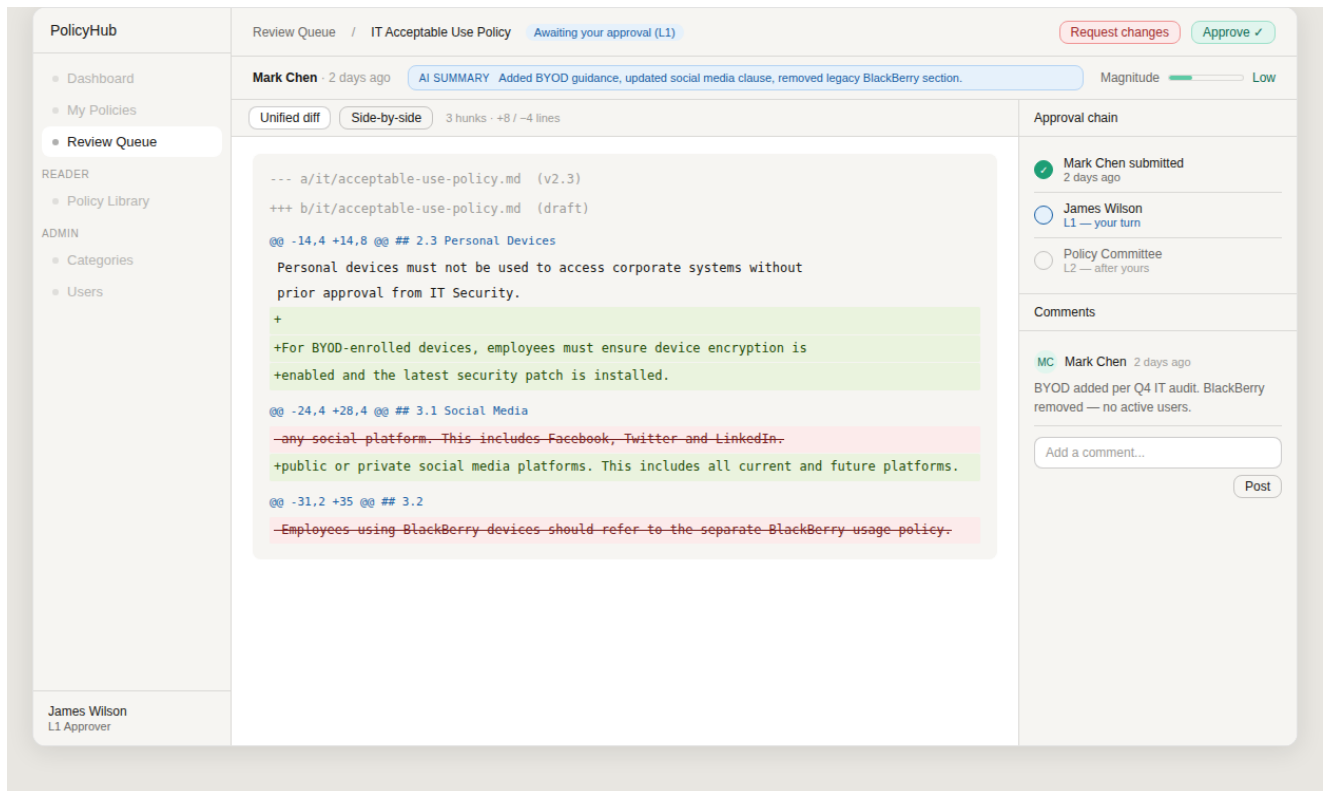


Fig. 04 — Review & Approval

The core review interface. Unified diff is the default view — a standard + / – line format that is readable once learned and technically straightforward to render (git diff output). A side-by-side rendered preview tab is available for users who prefer the formatted view. The AI summary and change magnitude bar appear in a sub-header so reviewers understand the scope before reading the diff. The approval chain panel on the right shows exactly where in the process this policy sits and whose turn it is. Comments are threaded per workflow instance.

Design decisions

- Unified diff is the default view — technically simple to render and consistent with developer tooling. Colleagues will be trained.
- Side-by-side rendered preview is a tab, not the default — available but not the entry point.
- Approve and Request Changes are the only workflow actions — no "reject" (always a request to change, never a dead-end).
- Approval chain panel shows full chain state — reviewer sees not just their step but the whole picture.
- Comments are scoped to the workflow instance, not the policy — a new review cycle starts fresh.

Reader Surface

05 — Policy Library (Portal)

Role: All staff (role-filtered)

Trigger: Clicking "Policy Library" in the sidebar

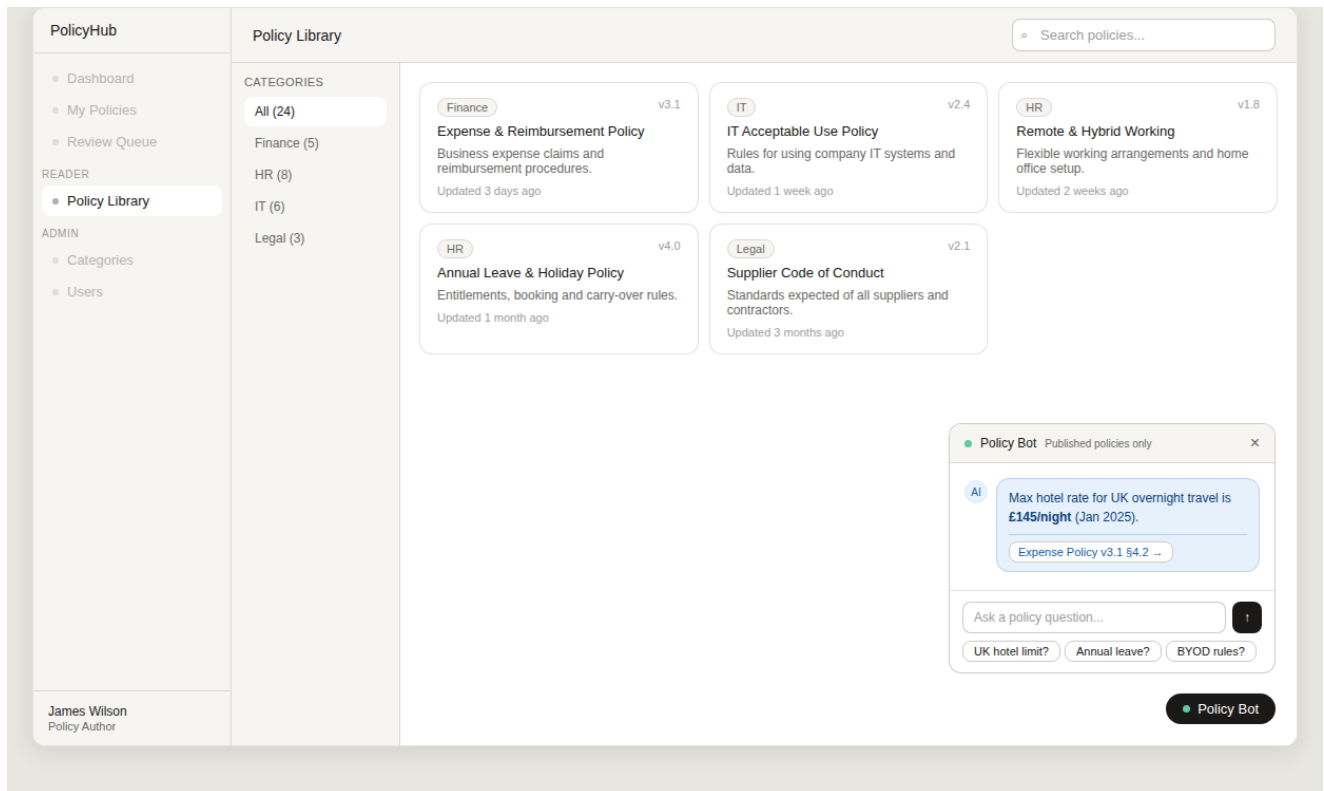


Fig. 05 — Policy Library (Portal)

The reader-facing policy library. Users see only policies tagged for their audience groups — missing policies return 404, not 403, to avoid leaking the existence of sensitive policies. Left sidebar provides category tree navigation. Full-text search is role-filtered at query time in Typesense. The Policy Bot FAB is persistent bottom-right. When open, the popover shows a citation-linked conversation; clicking a source pill loads the referenced policy into the main panel without closing the bot.

Design decisions

- RBAC is invisible — restricted policies simply do not exist in the UI, returning 404 not 403.
- Policy Bot uses the same FAB + popover pattern as the AI assistant in the editor — consistent interaction model across the product.
- Citation source pills load the referenced policy into the main panel while the bot conversation stays intact.
- Category tree is pre-filtered to show only categories containing policies the user can see.
- Card grid layout with search enables both browsing and targeted retrieval.

06 — Published Policy View

Role: All staff (permitted audience)

Trigger: Clicking a policy card in the library

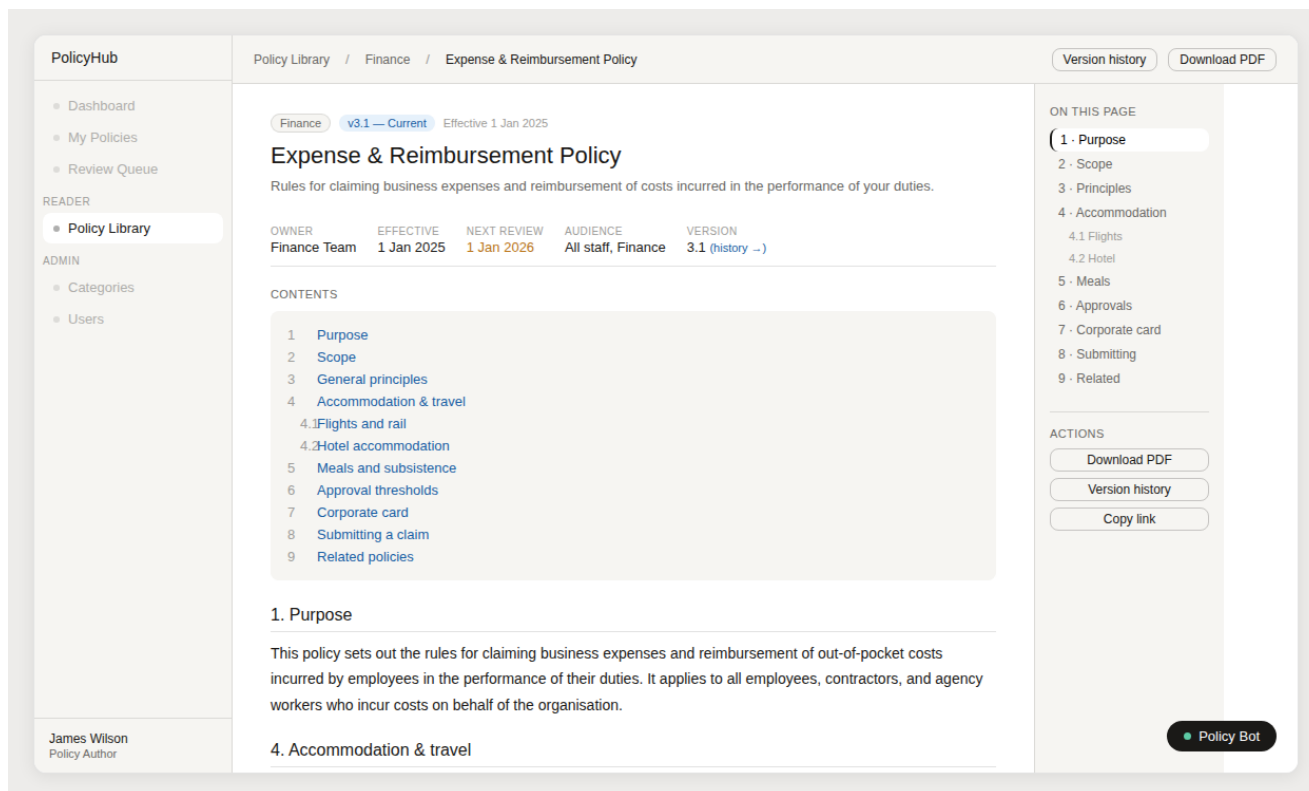


Fig. 06 — Published Policy View

The canonical published policy document. The publishing pipeline has injected the full boilerplate: version badge, metadata strip, auto-generated table of contents, policy body, related policies section, and version footer. A sticky right-hand "On this page" nav enables navigation within long policies. Review date shown in amber when approaching. Version history and PDF download are always accessible from topbar and the actions panel.

Design decisions

- ToC is auto-generated from markdown headings by the publishing pipeline — authors never write it manually.
- Metadata strip (owner, effective date, review date, audience) is injected from the policy database record, not the markdown.
- Related policies are rendered as linked cards pulled from `policy_related_policies` — authors tag relationships, the template renders them.
- "On this page" sticky nav appears for all policies — keeps orientation in long documents without a separate scroll-to-top button.
- Review date shown in amber when within 90 days of the due date — passive signal to owners without notifications.

07 — Version History

Role: All staff (permitted audience)

Trigger: Clicking "Version history" from the policy detail view or footer link

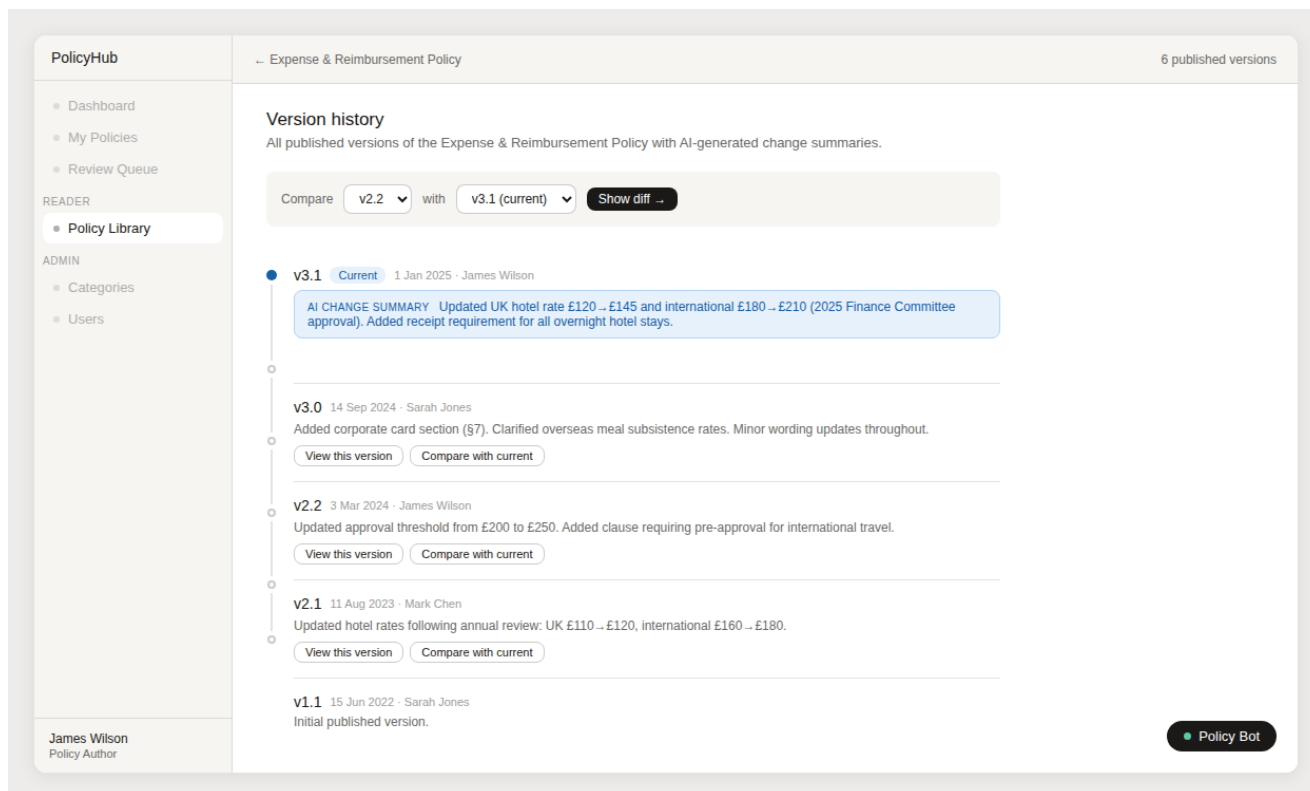


Fig. 07 — Version History

A chronological timeline of all published versions of a policy. Each entry shows the version number, publication date, author, and either an AI-generated change summary (for recent versions) or a brief human description. The current version always shows its AI summary expanded. A compare control at the top lets users select any two versions and jump directly to the diff view. All historical versions are accessible as read-only snapshots.

Design decisions

- AI change summary shown expanded on the current version — most readers come to understand what recently changed.
- Compare dropdowns are pre-populated: "from" defaults to the previous version, "to" defaults to current.
- Timeline spine visually communicates chronological ordering and makes the document's history feel like a story.
- All versions are permanently accessible — no archival or expiry of historical records.
- "View this version" loads the historical published HTML snapshot, not the current document.

08 — Version Diff (Reader)

Role: All staff (permitted audience)

Trigger: Clicking "Show diff" on Version History, or "Compare with current" on a version row

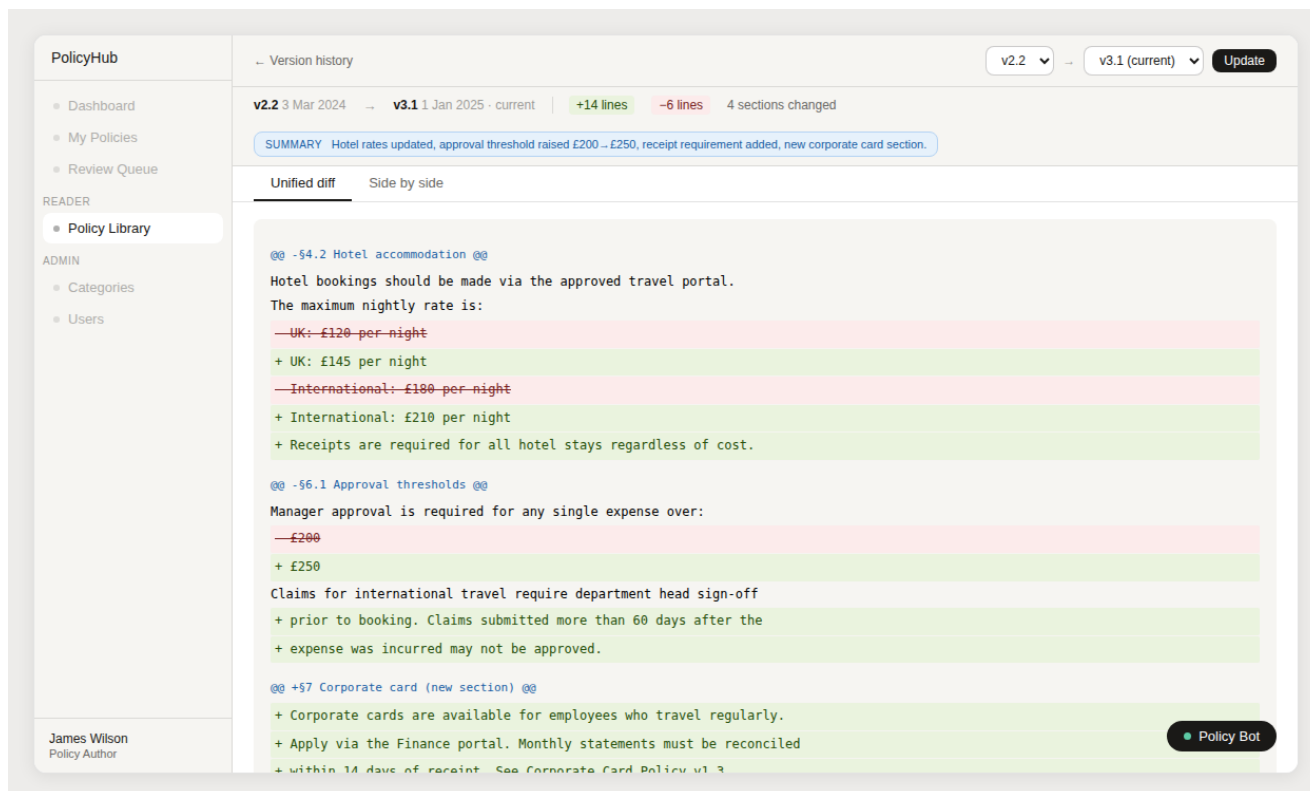


Fig. 08 — Version Diff (Reader)

Read-only version comparison for any two published versions of a policy. Uses the same unified diff / side-by-side tab pattern as the review screen for consistency. The stat strip shows line counts and the AI-generated summary before the reader engages with the diff detail. Version selectors in the topbar allow changing the comparison without returning to the history page. This view is accessible to all users permitted to read the policy.

Design decisions

- Identical tab pattern to the review diff — users learn the format once across both authoring and reading contexts.
- Read-only: no approval actions, no comment input — the UI surface is deliberately minimal.
- Stat strip (+N / -N line counts) gives immediate sense of change scale before scrolling.
- AI summary in stat strip provides plain-English context alongside the technical diff.
- Version selectors in topbar allow changing comparison on the fly without navigating back.

UX Journey Summary

Four primary journeys flow through the screens documented above.

Author: Draft to submission

Dashboard → Editor (new policy or edit draft) → Submit for Review. Author writes content in the WYSIWYG editor, addresses any reviewer comments inline, then pauses at the Submit screen to verify the AI change summary and assign reviewers before opening the workflow.

Reviewer / Approver: Review and decide

Dashboard (attention card) → Review screen. Reviewer reads the unified diff, checks the AI summary, reads or adds comments, then either Requests Changes (back to author) or Approves (advancing to the next approval level). Each approval level follows the same flow.

Reader: Find and read a policy

Policy Library → Policy Detail. Reader browses by category or searches by keyword. The policy renders with injected boilerplate (ToC, metadata, related policies). Reader can open the Policy Bot popover at any point to ask questions, with citation links loading the relevant policy into the main panel.

Reader: Understand what changed

Policy Detail → Version History → Version Diff. Reader navigates to the history page, selects two versions to compare, and reads the diff with the AI summary providing plain-English context. This journey is available to any user permitted to read the policy.

Next steps

- Admin surface screens (Users, Categories, IdP mappings, Audit log) — lower design risk, primarily table-and-form UI.
- Usability testing with policy authors (Milestone 1 gate) — validate editor UX assumptions with real users.
- Usability testing with reviewers (Milestone 2 gate) — validate unified diff comprehension.
- Policy Bot accuracy evaluation (Milestone 5 gate) — validate >85% correct citation rate before broad rollout.